movie 電影購票系統 API 規格書

2026-11-02

總覽

後端是 Spring Boot + MySQL `movie_pos`，所有 API 都在 `http://localhost:8080/api` 底下，回傳 JSON；React 開發伺服器（`http://localhost:5173`）已在 `WebConfig` 開放 CORS。

項目	說明
Base URL	`http://localhost:8080`
一般請求	Body 選 raw → JSON，Header `Content-Type: application/json`
電影新增 / 修改	`multipart/form-data`（電影 JSON + 圖片檔 + 預告片）
日期 / 時間	日期 `yyyy-MM-dd`、時間 `HH:mm:ss`、日期時間 `yyyy-MM-ddTHH:mm:ss`
圖片網址	回傳相對路徑，前端加上 Base URL：`http://localhost:8080/images/movies/1/poster1.jpg`
頁碼	page 從 1 開始，size 預設 10、最多 100

分頁回應 PageResponse

所有分頁查詢都回傳這個格式，資料放在 `content`。

```
{
  "content": [ { "...": "..." } ],
  "page": 1,
  "size": 10,
  "totalElements": 23,
  "totalPages": 3
}
```

錯誤回應

狀態碼	何時發生	回應內容
400	資料有誤、重複、座位已售出、JSON 格式錯誤、id 不是數字	`{"status":400,"error":"座位已售出：第 5 排第 3 號","timestamp":"..."}`
400	`@Valid` 欄位驗證失敗	`{"status":400,"errors":["片名不可以空白","請輸入片長"],"timestamp":"..."}`
401	會員登入帳號或密碼錯誤	`{"status":401,"error":"帳號或密碼錯誤"}`
404	找不到資料（XxxNotFoundException）	純文字，例如 `不存在，movie_id: 99`

狀態碼	何時發生	回應內容
404	分頁查詢沒有資料	空的 body
413	上傳檔案超過 100MB	`{"status":413,"error":"檔案太大，超過上傳上限"}`
415	multipart 的 JSON 欄位沒設 application/json	`{"status":415,"error":"Content-Type 不支援: ..."}`
500	檔案寫入失敗或未預期錯誤	`{"status":500,"error":"伺服器發生錯誤，請稍後再試"}`

電影 movie

電影不提供刪除，用 `isActive` 上下架：`true` = 上架（`is_active = 1`），`false` = 下架（0）。

方法	網址	用途	成功
GET	`/api/movie?page=1&size=10`	上架電影（分頁）	200
GET	`/api/movie/inactive?page=1&size=10`	下架電影（分頁）	200
GET	`/api/movie/{id}`	單一電影（含圖片）	200
POST	`/api/movie/add`	新增電影 + 圖片 + 預告片	201
PUT	`/api/movie/{id}`	修改電影；也用來上下架	200

GET /api/movie/1 回應

```
{
  "movieId": 1,
  "title": "星際探險隊",
  "durationMin": 128,
  "rating": "普遍級",
  "synopsis": "西元 2150 年，一支由五名年輕太空人組成的探險隊…",
  "director": "陳志遠",
  "actors": "林子翔, 許雅晴, 王大衛, 張美玲",
  "genre": "科幻",
  "releaseDate": "2026-09-18",
  "trailerUrl": "/videos/movies/1/trailer.mp4",
  "isActive": true,
  "updatedAt": "2026-11-01T09:51:21",
  "images": [
    { "imageId": 1, "movieId": 1, "imageUrl": "/images/movies/1/poster1.jpg",
      "imageType": "POSTER", "sortOrder": 0, "createdAt": "2026-11-01T09:51:21" },
    { "imageId": 2, "movieId": 1, "imageUrl": "/images/movies/1/still1.jpg",
      "imageType": "STILL", "sortOrder": 1, "createdAt": "2026-11-01T09:51:21" }
  ]
}
```

POST /api/movie/add（multipart/form-data）

Postman Body 選 form-data，打開 Content-Type 欄位（表格右上 ···）。

Key	類型	Content-Type	必填
`movie`	Text	`application/json`	是
`images`	Text	`application/json`	是，筆數 = files 檔案數
`files`	File（可多個）		是，至少 1 張；jpg / jpeg / png / webp
`trailer`	File		否；mp4 / webm

movie 的值：

```
{
  "title": "測試電影",
  "durationMin": 120,
  "rating": "普遍級",
  "synopsis": "測試用的電影",
  "director": "王小明",
  "actors": "演員 A, 演員 B",
  "genre": "動作",
  "releaseDate": "2026-12-01",
  "isActive": true
}
```

images 的值（順序對應 files）：

```
[ { "imageType": "poster" }, { "imageType": "still" } ]
```

成功回 201，內容同 GET `/api/movie/{id}`。圖片依類型改名為 `poster1.jpg`、`still1.png`...

PUT /api/movie/{id}（multipart/form-data）

只有 `movie` 必填；不換圖就不傳 `images`、`files`，不換預告片就不傳 `trailer`。下架只要把 movie 的 `isActive` 設成 `false`；沒傳 `isActive` 則維持原狀。

影廳、場次、票種

三者都是標準 CRUD；已被場次或票使用的資料，刪除時會被資料庫外鍵擋下，回 400。

方法	網址	用途
GET	`/api/hall?page=1&size=10`	全部影廳；加 hallName=第 1 廳 查單筆
GET	`/api/hall/search?keyword=1`	廳名模糊查詢
GET / PUT / DELETE	`/api/hall/{id}`	單筆查詢 / 修改 / 刪除
POST	`/api/hall`	新增影廳
POST	`/api/hall/with-showtimes`	一次新增影廳 + 場次（同一個交易）
GET	`/api/showtime?page=1&size=10`	全部場次；加 movieId=1 只看某部電影（含已過期場次）
GET	`/api/showtime/upcoming?movieId=1`	【新增】某部電影今天以後的場次（購票人用），依日期、時間排序，不分頁
GET / PUT / DELETE	`/api/showtime/{id}`	單筆查詢 / 修改 / 刪除
POST	`/api/showtime`	新增場次
GET	`/api/ticket-type?page=1&size=10`	全部票種

方法	網址	用途
GET / PUT / DELETE	`/api/ticket-type/{id}`	單筆查詢 / 修改 / 刪除
POST	`/api/ticket-type`	新增票種

POST /api/hall

規則：廳名必填、最多 20 字、不可重複；排數至少 10；第一排座位至少 1。

```
{ "hallName": "第 4 廳", "rowCount": 12, "firstRowSeats": 10 }
```

回應 201：

```
{ "hallId": 4, "hallName": "第 4 廳", "rowCount": 12, "firstRowSeats": 10 }
```

POST /api/hall/with-showtimes

影廳與場次在同一個交易裡新增：任何一筆失敗（廳名重複、電影不存在、放映日期早於上映日期、場次時間重複）就全部 rollback，影廳也不會存。場次不用傳 `hallId`，會自動屬於新影廳。

```
{
  "hall": { "hallName": "第 4 廳", "rowCount": 12, "firstRowSeats": 10 },
  "showtimes": [
    { "movieId": 1, "showDate": "2026-11-15", "startTime": "13:00:00" },
    { "movieId": 1, "showDate": "2026-11-15", "startTime": "16:30:00" },
    { "movieId": 2, "showDate": "2026-11-16", "startTime": "19:00:00" }
  ]
}
```

回應 201：

```
{
  "hall": { "hallId": 4, "hallName": "第 4 廳", "rowCount": 12, "firstRowSeats": 10 },
  "showtimes": [
    { "showtimeId": 7, "movieId": 1, "movieTitle": "星際探險隊", "hallId": 4,
      "hallName": "第 4 廳", "showDate": "2026-11-15", "startTime": "13:00:00" },
    { "showtimeId": 8, "movieId": 1, "movieTitle": "星際探險隊", "hallId": 4,
      "hallName": "第 4 廳", "showDate": "2026-11-15", "startTime": "16:30:00" },
    { "showtimeId": 9, "movieId": 2, "movieTitle": "午夜偵探", "hallId": 4,
      "hallName": "第 4 廳", "showDate": "2026-11-16", "startTime": "19:00:00" }
  ]
}
```

錯誤範例：`showtimes` 空白或 key 名稱寫錯（例如寫成 `showtimeList`）回 400 至少要有一個場次；`movieId` 不存在回 404。

測試資料要刪除時，先刪場次再刪影廳：

```
DELETE FROM movie_pos.showtime WHERE hall_id = 4;
DELETE FROM movie_pos.hall WHERE hall_id = 4;
```

POST /api/showtime

規則：電影、影廳要存在；放映日期不能早於上映日期；同一影廳、同一日期時間只能一場。

```
{ "movieId": 1, "hallId": 3, "showDate": "2026-11-15", "startTime": "16:30:00" }
```

回應 201：

```
{
  "showtimeId": 7,
```

```
  "movieId": 1,
  "movieTitle": "星際探險隊",
  "hallId": 3,
  "hallName": "第 3 廳",
  "showDate": "2026-11-15",
  "startTime": "16:30:00"
}
```

GET /api/showtime/upcoming?movieId=1【新增】

電影介紹頁用：只回放映日期是今天或以後的場次，依日期、開演時間排序。回傳的是陣列，不是分頁格式；沒有場次（或 movieId 不存在）回 200 + [ ]。

```
[
  { "showtimeId": 7, "movieId": 1, "movieTitle": "星際探險隊", "hallId": 3,
    "hallName": "第 3 廳", "showDate": "2026-11-15", "startTime": "13:00:00" },
  { "showtimeId": 8, "movieId": 1, "movieTitle": "星際探險隊", "hallId": 3,
    "hallName": "第 3 廳", "showDate": "2026-11-15", "startTime": "16:30:00" }
]
```

注意：今天已經開演的場次仍會出現。`movieId` 必填；沒帶時 `GlobalExceptionHandler` 目前沒有處理缺少參數的例外，會回 500。

POST /api/ticket-type

`typeId` 由資料庫自動編號，不用傳。`basePrice` 是實際售價，`discountRate` 介於 0.01 ~ 1.00。

```
{ "typeName": "敬老票", "basePrice": 150, "discountRate": 0.50,
  "description": "一般票價格*5 折" }
```

回應 201：

```
{ "typeId": 6, "typeName": "敬老票", "basePrice": 150, "discountRate": 0.50,
  "description": "一般票價格*5 折" }
```

購票：訂單 orders 與票 ticket

一次購票 = 一筆訂單 + 每個座位一張票，金額全由後端計算；訂單只能新增與查詢，票只能查詢。

方法	網址	用途
POST	`/api/orders`	購票
GET	`/api/orders?page=1&size=10`	全部訂單（管理者）；加 memberId=1 查自己的訂單
GET	`/api/orders/{id}`	單筆訂單（含票）
GET	`/api/ticket?showtimeId=1`	某場次已售出座位（畫座位圖用）；沒賣出回 []
GET	`/api/ticket?orderId=1`	某訂單的票
GET	`/api/ticket/{id}`	單一張票

價格規則

欄位	算法	例：VIP 票（base 150、折扣 0.50）
`ticket.price`	base_price	150

欄位	算法	例：VIP 票（base 150、折扣 0.50）
ticket.original_price	base_price ÷ discount_rate，四捨五入	300
orders.original_amount	票的 original_price 加總	
orders.total_amount	票的 price 加總	
orders.discount_amount	original_amount − total_amount	

POST /api/orders

檢查：會員、場次存在；至少 1 張；排數在影廳範圍內；座位不重複、未售出；票種存在。任一項不過回 400（會員、場次不存在回 404），訂單與票都不存。memberId 由登入回應取得。

```
{
  "memberId": 1,
  "showtimeId": 2,
  "tickets": [
    { "rowNo": 5, "seatNo": 3, "typeId": 2 },
    { "rowNo": 5, "seatNo": 4, "typeId": 1 }
  ]
}
```

回應 201（orderTime 由資料庫產生，存檔後重新讀回，所以會有值）：

```
{
  "orderId": 6,
  "memberId": 1,
  "showtimeId": 2,
  "originalAmount": 600,
  "discountAmount": 150,
  "totalAmount": 450,
  "orderTime": "2026-11-01T11:05:12",
  "tickets": [
    { "ticketId": 11, "orderId": 6, "showtimeId": 2, "rowNo": 5, "seatNo": 3,
      "typeId": 2, "typeName": "一般票", "originalPrice": 300, "price": 300 },
    { "ticketId": 12, "orderId": 6, "showtimeId": 2, "rowNo": 5, "seatNo": 4,
      "typeId": 1, "typeName": "VIP 票", "originalPrice": 300, "price": 150 }
  ],
  "movieTitle": "星際探險隊",
  "hallName": "第 1 廳",
  "showDate": "2026-11-10",
  "startTime": "13:00:00"
}
```

【新增】movieTitle、hallName、showDate、startTime 給「我的訂單」頁直接顯示，不用再查場次。這 4 個欄位沒有寫在 OrdersResponse 的 @JsonPropertyOrder 裡，所以排在 tickets 後面。GET /api/orders、GET /api/orders/{id} 的每筆訂單也是這個格式。

座位已售出時回 400：

```
{ "status": 400, "error": "座位已售出：第 5 排第 3 號", "timestamp": "2026-11-01T11:06:00" }
```

GET /api/ticket?showtimeId=1 回應

```
[
  { "ticketId": 1, "orderId": 1, "showtimeId": 1, "rowNo": 5, "seatNo": 4,
    "typeId": 2, "typeName": "一般票", "originalPrice": 300, "price": 300 },
  { "ticketId": 2, "orderId": 1, "showtimeId": 1, "rowNo": 5, "seatNo": 5,
```

```
    "typeId": 2, "typeName": "一般票", "originalPrice": 300, "price": 300 }
]
```

管理者報表

兩支報表都只有查詢；金額是實收（`ticket.price`）。下面的數字是 2026-11-01 重建的 5 筆訂單、以 2026-11-01 當天查詢的結果。

方法	網址	用途
GET	`/api/admin/report/movie-sales`	全部電影的銷售張數與金額（沒賣出的也列出，為 0）
GET	`/api/admin/report/movie-sales?title=星際`	片名包含「星際」的電影；找不到回 404
GET	`/api/admin/report/sales?period=week`	本週（星期一 ~ 星期日）銷售總表
GET	`/api/admin/report/sales?period=month`	本月（1 號 ~ 月底）銷售總表；period 其他值回 400

GET /api/admin/report/movie-sales

依銷售金額由高到低排序。

```
[
  { "movieId": 1, "title": "星際探險隊", "ticketCount": 4, "totalAmount": 1020 },
  { "movieId": 2, "title": "午夜偵探", "ticketCount": 3, "totalAmount": 720 },
  { "movieId": 4, "title": "時光咖啡館", "ticketCount": 2, "totalAmount": 570 },
  { "movieId": 3, "title": "海洋之歌", "ticketCount": 1, "totalAmount": 540 },
  { "movieId": 5, "title": "鋼鐵守護者", "ticketCount": 0, "totalAmount": 0 }
]
```

GET /api/admin/report/sales?period=month

期間依訂單的下單時間 `order_time` 計算。`daily` 只列有訂單的日期，`movies` 只列有賣出的電影。

```
{
  "period": "month",
  "startDate": "2026-11-01",
  "endDate": "2026-11-30",
  "orderCount": 5,
  "ticketCount": 10,
  "originalAmount": 3300,
  "discountAmount": 450,
  "totalAmount": 2850,
  "daily": [
    { "saleDate": "2026-11-01", "orderCount": 2, "ticketCount": 4, "totalAmount": 1020 }
,
    { "saleDate": "2026-11-02", "orderCount": 2, "ticketCount": 4, "totalAmount": 1260 }
,
    { "saleDate": "2026-11-03", "orderCount": 1, "ticketCount": 2, "totalAmount": 570 }
  ],
  "movies": [
    { "movieId": 1, "title": "星際探險隊", "ticketCount": 4, "totalAmount": 1020 },
    { "movieId": 2, "title": "午夜偵探", "ticketCount": 3, "totalAmount": 720 },
    { "movieId": 4, "title": "時光咖啡館", "ticketCount": 2, "totalAmount": 570 },
```

```
    { "movieId": 3, "title": "海洋之歌", "ticketCount": 1, "totalAmount": 540 }
  ]
}
```

period=week 格式相同；2026-11-01 是星期日，本週為 2026-10-26 ~ 2026-11-01，只含訂單 1、2（實收 1020）。

會員 member、管理者 admin

密碼以 SHA-256 加密後儲存，回應不含密碼；帳號重複回 409（會員）或 400（管理者）。會員登入 API 已完成；管理者登入與權限保護尚未實作。

方法	網址	用途
GET	/api/member?page=1&size=10	全部會員
GET	/api/member/search?keyword=wang&page=1&size=10	帳號 / 姓名 / 電話 / Email 模糊查詢
POST	/api/member	註冊會員
POST	/api/member/login	【新增】會員登入；成功回會員資料，失敗回 401
PUT	/api/member/{id}	修改會員；password 留空則不改密碼
DELETE	/api/member/{id}	刪除會員
GET	/api/admin?page=1&size=10	全部管理者；加 username=admin 查單筆
GET	/api/admin/search?keyword=a	帳號 / 姓名模糊查詢
POST	/api/admin	新增管理者
PUT	/api/admin/{id}	修改管理者
DELETE	/api/admin/{username}	刪除管理者（用帳號）

POST /api/member

```
{
  "username": "wang01",
  "password": "Abc12345",
  "name": "王小明",
  "phone": "0912345678",
  "email": "wang01@example.com"
}
```

回應 201：

```
{ "memberId": 2, "username": "wang01", "name": "王小明",
  "phone": "0912345678", "email": "wang01@example.com" }
```

POST /api/member/login【新增】

比對帳號，並把密碼做 SHA-256 後和資料庫比較。為了不讓人試出哪些帳號存在，帳號不存在和密碼錯誤都回同一個訊息。

```
{ "username": "wang01", "password": "Abc12345" }
```

回應 200（內容同註冊回應，不含密碼）：

```
{ "memberId": 2, "username": "wang01", "name": "王小明",
  "phone": "0912345678", "email": "wang01@example.com" }
```

情況	狀態碼	回應內容
帳號密碼正確	200	會員資料（如上）
帳號不存在或密碼錯誤	401	`{"status":401,"error":"帳號或密碼錯誤"}`
帳號或密碼沒填	400	`{"status":400,"errors":["請輸入帳號"],"timestamp":"..."}`

前端登入成功後把 `memberId` 存在 localStorage，購票（`POST /api/orders`）與查訂單（`GET /api/orders?memberId=`）都用它。目前後端沒有 Session / Token 驗證，`memberId` 由前端傳入。

POST /api/admin

```
{ "username": "admin02", "password": "Admin1234", "name": "店長" }
```

回應 201：

```
{ "adminId": 2, "username": "admin02", "name": "店長" }
```

建議測試順序

依資料相依順序測，前一步的 id 是下一步的輸入。
`movie_assets/postman/movie_pos_postman_collection_2026-11-01.json` 可在 Postman 按 Import 匯入全部請求。

1. `GET /api/ticket-type`、`GET /api/hall` — 確認基礎資料
2. `GET /api/movie` — 取得電影 id
3. `GET /api/showtime/upcoming?movieId=1` — 取得今天以後的場次 id
4. `GET /api/ticket?showtimeId=1` — 看哪些座位已售出
5. `POST /api/member/login` — 登入取得 memberId；再送一次錯誤密碼，應回 401
6. `POST /api/orders` — 買未售出的座位；確認回應有 orderTime、片名與場次時間；再送一次同樣內容，應回 400「座位已售出」
7. `GET /api/orders?memberId=1` — 確認新訂單與金額
8. `GET /api/admin/report/movie-sales`、`GET /api/admin/report/sales?period=month` — 確認報表數字跟著增加
9. `POST /api/showtime`、`POST /api/ticket-type`、`POST /api/hall` — 後台維護功能